NAME:M.RAMAKRISHNA RAJU
STUDENT ID:2400031163

DEPARTMENT OF CSE, CS&IT, AI&DS
COURSE CODE: 24SDCS02E FULL STACK APPLICATION DEVELOPMENT

Date of the Session:7-01-2026

#SKILL - 3 Working with HQL - Sorting, Pagination & Aggregates

Prerequisites:

 Basic knowledge of Hibernate

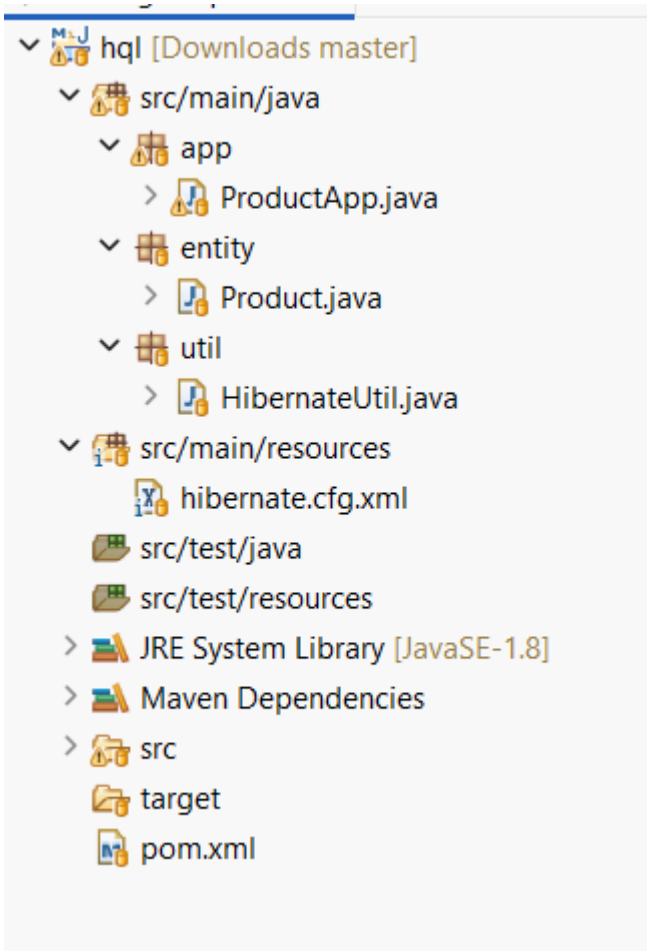
 Basic idea of HQL syntax

In this lab, you will extend the Hibernate module created in **Skill 2** and perform advanced data
retrieval using **HQL**, including **sorting**, **pagination**, **filtering**, and **aggregate functions**. These
operations help in retrieving structured and meaningful data from the database using Hibernate.

Tasks:

1. Use the **Product entity** created in Skill 2.
2. Add **5–8 additional Product records** into the database.
3. Write HQL queries to retrieve all products sorted by price:
a. Ascending order
b. Descending order
4. Write an HQL query to sort products by quantity (highest first).
5. Implement pagination using HQL to display:
a. First 3 products
b. Next 3 products
6. Write HQL queries for aggregate operations:
a. Count total number of products
b. Count products where quantity > 0
c. Count products grouped by description
d. Find minimum and maximum price
7. Write an HQL query using **GROUP BY** to group products by description.
8. Write an HQL query using **WHERE** to filter products within a price range.
9. Write HQL queries using **LIKE** such as:
a. Names **starting with** certain letters
b. Names **ending with** certain letters
c. Names containing a pattern **anywhere** (substring)
d. Names with an **exact character length**
10. Push the Hibernate project to GitHub.

Structure:



Codes:
ProductApp.java

```
package app;

import java.util.List;

import org.hibernate.Session;
import org.hibernate.Transaction;
import org.hibernate.query.Query;

import entity.Product;
import util.HibernateUtil;

public class ProductApp {

    public static void main(String[] args) {

        Session session = HibernateUtil.getSessionFactory().openSession();
        Transaction tx = session.beginTransaction();

        // INSERT SAMPLE DATA
        session.save(new Product("Laptop", "Electronics", 65000, 10));
```

```
session.save(new Product("Mouse", "Electronics", 800, 50));
session.save(new Product("Keyboard", "Electronics", 1500, 30));
session.save(new Product("Chair", "Furniture", 3500, 15));
session.save(new Product("Table", "Furniture", 7000, 5));
session.save(new Product("Pen", "Stationery", 20, 0));
session.save(new Product("Notebook", "Stationery", 100, 100));

// SORTING
session.createQuery("FROM Product p ORDER BY p.price ASC", Product.class)
    .list().forEach(System.out::println);

session.createQuery("FROM Product p ORDER BY p.price DESC", Product.class)
    .list().forEach(System.out::println);

session.createQuery("FROM Product p ORDER BY p.quantity DESC", Product.class)
    .list().forEach(System.out::println);

// PAGINATION
session.createQuery("FROM Product", Product.class)
    .setFirstResult(0)
    .setMaxResults(3)
    .list().forEach(System.out::println);

session.createQuery("FROM Product", Product.class)
    .setFirstResult(3)
    .setMaxResults(3)
    .list().forEach(System.out::println);

// AGGREGATES
Long total =
    session.createQuery("SELECT COUNT(p) FROM Product p", Long.class)
        .uniqueResult();
System.out.println("Total Products: " + total);

Long available =
    session.createQuery(
        "SELECT COUNT(p) FROM Product p WHERE p.quantity > 0",
        Long.class)
        .uniqueResult();
System.out.println("Available Products: " + available);

Object[] minMax =
    session.createQuery(
        "SELECT MIN(p.price), MAX(p.price) FROM Product p",
        Object[].class)
        .uniqueResult();
System.out.println("Min Price: " + minMax[0]);
System.out.println("Max Price: " + minMax[1]);

// GROUP BY
List<Object[]> grouped =
    session.createQuery(
        "SELECT p.description, COUNT(p) FROM Product p GROUP BY p.description",
        Object[].class)
        .list();

for (Object[] row : grouped) {
```

```
        System.out.println(row[0] + " -> " + row[1]);
    }

    // WHERE
    session.createQuery(
        "FROM Product p WHERE p.price BETWEEN 1000 AND 10000",
        Product.class)
        .list().forEach(System.out::println);

    // LIKE
    session.createQuery("FROM Product p WHERE p.name LIKE 'L%'", Product.class)
        .list().forEach(System.out::println);

    session.createQuery("FROM Product p WHERE p.name LIKE '%r'", Product.class)
        .list().forEach(System.out::println);

    session.createQuery("FROM Product p WHERE p.name LIKE '%tab%'", Product.class)
        .list().forEach(System.out::println);

    session.createQuery("FROM Product p WHERE LENGTH(p.name)=5", Product.class)
        .list().forEach(System.out::println);

    tx.commit();
    session.close();
    HibernateUtil.shutdown();
  }
}
```

Product.java:

```java
package entity;

import javax.persistence.*;

@Entity
@Table(name = "product")
public class Product {

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private int id;

private String name;
private String description;
private double price;
private int quantity;

public Product() {}

public Product(String name, String description, double price, int quantity)
{
this.name = name;
this.description = description;
this.price = price;
this.quantity = quantity;
}
```

```
public int getId() { return id; }
public void setId(int id) { this.id = id; }

public String getName() { return name; }
public void setName(String name) { this.name = name; }

public String getDescription() { return description; }
public void setDescription(String description) { this.description =
description; }

public double getPrice() { return price; }
public void setPrice(double price) { this.price = price; }

public int getQuantity() { return quantity; }
public void setQuantity(int quantity) { this.quantity = quantity; }

@Override
public String toString() {
return id + " | " + name + " | " + description + " | " + price + " | " +
quantity;
}
}
```

HibernateUtil.java:

```
package util;

import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;

public class HibernateUtil {

   private static final SessionFactory sessionFactory = buildSessionFactory();

   private static SessionFactory buildSessionFactory() {
      try {
         return new Configuration().configure().buildSessionFactory();
      } catch (Throwable ex) {
         throw new ExceptionInInitializerError(ex);
      }
   }

   public static SessionFactory getSessionFactory() {
      return sessionFactory;
   }

   public static void shutdown() {
      getSessionFactory().close();
   }
}
```

Hibernate.cfg,xml:

```
<hibernate-configuration>
```

```
  <session-factory>

    <property name="hibernate.connection.driver_class">com.mysql.cj.jdbc.Driver</property>
    <property name="hibernate.connection.url">
      jdbc:mysql://localhost:3306/db1
    </property>
    <property name="hibernate.connection.username">root</property>
    <property name="hibernate.connection.password">Mahesh@31164</property>

    <property name="hibernate.dialect">org.hibernate.dialect.MySQL8Dialect</property>
    <property name="hibernate.show_sql">true</property>
    <property name="hibernate.hbm2ddl.auto">update</property>

    <mapping class="entity.Product"/>

  </session-factory>
</hibernate-configuration>
```

pom.xml:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">

<modelVersion>4.0.0</modelVersion>

<groupId>com.example</groupId>
<artifactId>hql</artifactId>
<version>1.0</version>

<dependencies>

<dependency>
<groupId>org.hibernate</groupId>
<artifactId>hibernate-core</artifactId>
<version>5.6.15.Final</version>
</dependency>

<dependency>
<groupId>mysql</groupId>
<artifactId>mysql-connector-java</artifactId>
<version>8.0.33</version>
</dependency>

<dependency>
<groupId>javax.persistence</groupId>
<artifactId>javax.persistence-api</artifactId>
<version>2.2</version>
</dependency>

</dependencies>
</project>
```

Output:

```
MySQL 8.0 Command Line Cl
    -> ;
+---------------+
| Tables_in_db1 |
+---------------+
| student       |
+---------------+
1 row in set (0.07 sec)

mysql> show tables;
+---------------+
| Tables_in_db1 |
+---------------+
| product       |
| student       |
+---------------+
2 rows in set (0.00 sec)

mysql> select * from product;
+----+---------------+--------+-------+----------+
| id | description   | name   | price | quantity |
+----+---------------+--------+-------+----------+
|  1 | Gaming Laptop | Laptop | 70000 |        8 |
+----+---------------+--------+-------+----------+
1 row in set (0.00 sec)

mysql> select * from product;
+----+---------------+----------+-------+----------+
| id | description   | name     | price | quantity |
+----+---------------+----------+-------+----------+
|  1 | Gaming Laptop | Laptop   | 70000 |        8 |
|  3 | Electronics   | Laptop   | 65000 |       10 |
|  4 | Electronics   | Mouse    |   800 |       50 |
|  5 | Electronics   | Keyboard |  1500 |       30 |
|  6 | Furniture     | Chair    |  3500 |       15 |
|  7 | Furniture     | Table    |  7000 |        5 |
|  8 | Stationery    | Pen      |    20 |        0 |
|  9 | Stationery    | Notebook |   100 |      100 |
+----+---------------+----------+-------+----------+
8 rows in set (0.00 sec)

mysql> select count(8
```

```
MySQL 8.0 Command Line Cl
| id | description   | name     | price | quantity |
+----+---------------+----------+-------+----------+
|  1 | Gaming Laptop | Laptop   | 70000 |        8 |
|  3 | Electronics   | Laptop   | 65000 |       10 |
|  4 | Electronics   | Mouse    |   800 |       50 |
|  5 | Electronics   | Keyboard |  1500 |       30 |
|  6 | Furniture     | Chair    |  3500 |       15 |
|  7 | Furniture     | Table    |  7000 |        5 |
|  8 | Stationery    | Pen      |    20 |        0 |
|  9 | Stationery    | Notebook |   100 |      100 |
+----+---------------+----------+-------+----------+
8 rows in set (0.00 sec)

mysql> select count(8
    -> select count(*) from product;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 's
elect count(*) from product' at line 2
mysql> SELECT COUNT(*) FROM product;
+----------+
| COUNT(*) |
+----------+
|        8 |
+----------+
1 row in set (0.00 sec)

mysql> SELECT * FROM product ORDER BY price ASC;
+----+---------------+----------+-------+----------+
| id | description   | name     | price | quantity |
+----+---------------+----------+-------+----------+
|  8 | Stationery    | Pen      |    20 |        0 |
|  9 | Stationery    | Notebook |   100 |      100 |
|  4 | Electronics   | Mouse    |   800 |       50 |
|  5 | Electronics   | Keyboard |  1500 |       30 |
|  6 | Furniture     | Chair    |  3500 |       15 |
|  7 | Furniture     | Table    |  7000 |        5 |
|  3 | Electronics   | Laptop   | 65000 |       10 |
|  1 | Gaming Laptop | Laptop   | 70000 |        8 |
+----+---------------+----------+-------+----------+
8 rows in set (0.00 sec)

mysql>
```

VIVA QUESTIONS:

1. What is Dependency Injection (DI) in Spring?

Ans: Dependency Injection (DI) is a core concept of Spring where objects are provided with their dependencies by the Spring container, instead of creating them manually using the new keyword.

2. What is the difference between Constructor DI and Setter DI?

Ans: Difference between Constructor DI and Setter DI

Injection Method:

- Constructor DI: Dependencies are injected through the **constructor**
- Setter DI: Dependencies are injected through **setter methods**

Mandatory Dependency:

- Constructor DI: Used when the dependency is **mandatory**
- Setter DI: Used when the dependency is **optional**

3. What is the purpose of the IoC container?

Ans: The IoC (Inversion of Control) container is responsible for:

- Creating objects (beans)
- Injecting dependencies
- Managing bean lifecycle
- Configuring beans

4. Why is DI preferred over manual object creation?

Ans: DI is preferred because it:

- Reduces tight coupling.
- Improves testability

5. Can Constructor Injection and Setter Injection be used together?

Ans: **Yes**, both can be used together in the same class.

When to use together:

- Constructor DI → mandatory dependencies
- Setter DI → optional dependencies